HW11_DSA8420

April 14, 2025

Primary Task: Set up a (mixed) integer linear programming model for each of the following problems. Make sure you state the definition of the variables. Do not solve the problem.

[1]:
```
# libraries
import numpy as np
from scipy.optimize import linprog
```

---

1 (10 points) A chair manufacturer makes three different types of chairs, each of which must go through *sanding, staining, and varnishing.* In addition, the *model with the vinyl-covered back and seat must go through an upholstering process.* Table 1 gives the time required for each operation on each type of chair, the available time for each operation in hours per month, and the profit per chair for each model. How many chairs of each type should be made to maximize the total profit?

Table 1

Model	Sanding(hrs)	Staining(hrs)	Varnishing(hrs)	Upholstering(hrs)	Profit($)
A-solid back & seat	1.0	0.5	0.7	0	10
B-ladder back, solid seat	1.2	0.5	0.7	0	13
C-Vinyl back & seat	0.7	0.3	0.3	0.7	8
Net available time per month	600	300	300	140	

Answer:

We will define our decision variables are as follows...

x_A: # of Model A chairs to make.

x_B: # of Model B chairs to make

x_C: # of Model C chairs to make

We can define our *Objective Function* to maximize total profit using our decision variables as...

$$MaxZ = 10x_A + 13x_B + 8x_C$$

Subject to Constraints:

Sanding Time Constraint

$$1.0x_A + 1.2x_B + 0.7x_C \leq 600$$

Staining Time Constraint

$$0.5x_A + 0.5x_B + 0.3x_C \leq 300$$

Varnishing Time Constraint

$$0.7x_A + 0.7x_B + 0.3x_C \leq 300$$

Upholstering Time Constraint

$$0.7x_C \leq 140$$

[2]:
```
""" Problem 1 setup """
# max 10x_A + 13x_B + 8x_C ; flipping coeff since linprog does min instead of max
c = -np.array([10, 13, 8])

# Constraints matrix
A_ub = np.array([
    [1.0, 1.2, 0.7],
    [0.5, 0.5, 0.3],
    [0.7, 0.7, 0.3],
    [0.0, 0.0, 0.7]
])
b_ub = np.array([600, 300, 300, 140]) # RHS
bounds = 3*[(0, None)]
```

---

2 (10 points) A company is considering several investment opportunities, each of which differs in the initial capital required and cannot be invested repeatedly. The accounting department has done a thorough analysis of each of these investments and has estimated the long-term profit of each. The initial capital required and estimated profits (in millions of dollars) are summarized in Table 2. *Investments 1 and 4 are considered high-risk investments and management has decided to invest in at most one of these.* In addition, *Investment 6 is contingent upon also investing in Investment 3*, that is, Investment 6 is invested only if Investment 3 is invested. If $120 million of initial capital is available, formulate an integer programming model to determine the optimal investment strategy.

Table 2

Investment	Initial Capital	Estimated Profit
1	26	18
2	34	12
3	18	7
4	45	24
5	31	11
6	39	15
7	23	9
8	13	6

Answer:

Unlike our previous question, we are now working with binary decision variables for investment i where i is one of 8 investments. I will define our binary decision variables are as follows.

Let $x_i \in 0, 1$ for investment i, where...

$x_i == 1$: Investment i chosen.

$x_i == 0$: Investment i not chosen.

We can define our *Objective Function* to maximize net estimated profit using our binary decision variables as...

$$MaxZ = 18x_1 + 12x_2 + 7x_3 + 24x_4 + 11x_5 + 15x_6 + 9x_7 + 6x_8$$

Subject to Constraints:

Binary Variables Constraint

$x_i \in [0, 1]$ for investment $i \in [1, 2, 3, 4, 5, 6, 7, 8]$

Capital Constraint

"If $120 million of initial capital is available, formulate an integer programming model to det

$$26x_1 + 34x_2 + 18x_3 + 45x_4 + 31x_5 + 39x_6 + 23x_7 + 13x_8 \leq 120$$

High Risk Constraint

"Investments 1 and 4 are considered high-risk investments and management has decided to invest

$$x_1 + x_4 \leq 1$$

Contingancy Constraint

"Investment 6 is contingent upon also investing in Investment 3."

$$x_6 \leq x_3$$

[3]:
```
""" Problem 2 setup """
# flip obj func since linprog does min instead of max
c = -np.array([18, 12, 7, 24, 11, 15, 9, 6])

# Binary var constraint
bounds = 8 * [(0,1)]

# Capital constraints
A_ub = [np.array([26, 34, 18, 45, 31, 39, 23, 13])] # nest constraints in A_ub
b_ub = [120]  # next capital available

# High risk constraints
A_ub.append([1, 0, 0, 1, 0, 0, 0, 0])
b_ub.append(1)

# Contingency Constraints
A_ub.append([0, 0, -1, 0, 0, 1, 0, 0])
b_ub.append(0)

# scipy linprog setup
A_ub = np.array(A_ub)
b_ub = np.array(b_ub)

print("A_ub = ", A_ub)
print("b_ub = ", b_ub)
```

```
A_ub =  [[26 34 18 45 31 39 23 13]
 [ 1  0  0  1  0  0  0  0]
 [ 0  0 -1  0  0  1  0  0]]
b_ub =  [120   1   0]
```

---

3 (10 points) A city is trying to establish a municipal emergency ambulance service that can adequately service all sections of the city. The planning committee has established *eight potential sites* for the emergency service facilities. However, due to time and distance requirements,*each potential site can provide coverage to only a subset of the city's six sections.* Table 3 summarizes the sections for which each site provides coverage. Formulate an integer programming model for determining the fewest number of facilities that will provide all sections of the city with adequate coverage.

Table 3

Site	Section of City Covered
1	A, B, E
2	C, D
3	B, C, D
4	A, D, F
5	B, F
6	A, D, E, F
7	A, C, E
8	B, D, F

Answer:

Binary Decision Variables:

Let $x_f \in 0, 1$ for facility f, where…

$x_f == 1$: Facility f chosen.

$x_f == 0$: Facility f not chosen.

Objective Function:

$$MinZ = x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8$$

Subject to Constraints:

Binary Variables Constraint

$x_f \in [0, 1]$ for facility $f \in [1, 2, 3, 4, 5, 6, 7, 8]$

Section A Constraint

`"Covered by sites: 1, 4, 6, 7."`

$$x_1 + x_4 + x_6 + x_7 \geq 1$$

Section B Constraint

"Covered by sites: 1, 3, 5, 8."

$$x_1 + x_3 + x_5 + x_8 \geq 1$$

Section C Constraint

"Covered by sites: 2, 3, 7."

$$x_2 + x_3 + x_7 \geq 1$$

Section D Constraint

"Covered by sites: 2, 3, 4, 6, 8."

$$x_2 + x_3 + x_4 + x_6 + x_8 \geq 1$$

Section E Constraint

"Covered by sites: 1, 6, 7."

$$x_1 + x_6 + x_7 \geq 1$$

Section F Constraint

"Covered by sites: 4, 5, 6, 8."

$$x_4 + x_5 + x_6 + x_8 \geq 1$$

[4]:
```
""" Problem 3 setup """
# obj func
c = np.ones(8)

# Section constraints + transformed Ax >= 1 to -Ax <= -1
A_ub = -np.array([
    [-1,  0,  0, -1,  0, -1, -1,  0],  # A: 1, 4, 6, 7
    [-1,  0, -1,  0, -1,  0,  0, -1],  # B: 1, 3, 5, 8
    [ 0, -1, -1,  0,  0,  0, -1,  0],  # C: 2, 3, 7
    [ 0, -1, -1, -1,  0, -1,  0, -1],  # D: 2, 3, 4, 6, 8
    [-1,  0,  0,  0,  0, -1, -1,  0],  # E: 1, 6, 7
    [ 0,  0,  0, -1, -1, -1,  0, -1],  # F: 4, 5, 6, 8
])
b_ub = -np.ones(6) # RHS

# Binary vars constraint
bounds = 8*[(0, 1)]
```

---

4 (10 points) Distributed oil deposits are often tapped by drilling from a central site. BlackGold Corp. is considering two potential drilling sites for reaching four targets (possible deposits). Table 4 provides the preparation costs (in millions of dollars) at each of the two sites and the cost of drilling from each site to each deposit. Formulate the problem of choosing which sites to open and which sites should tap each deposit as an integer program.

Table 4

	Drilling Cost To				
Site	Target 1	Target 2	Target 3	Target 4	Preparation Cost
1	2	1	8	5	5
2	4	6	3	1	6

Answer:

Binary Decision Variables:

Let $y_s \in 0, 1$ for Site s, where...

$y_s == 1$: Site s is prepared.

$y_s == 0$: Site s is not prepared.

for $s \in [1, 2]$.

Let $x_{s,t} \in 0, 1$ for Target t and Site s, where...

$x_{s,t} == 1$: Target t is drilled from Site s.

$x_{s,t} == 0$: Target t is not drilled from Site s.

for $s \in [1, 2], t \in [1, 2, 3, 4]$.

Objective Function:

$MinimizeZ = 5y_1 + 6y_2 + 2x_{1,1} + 1x_{1,2} + 8x_{1,3} + 5x_{1,4} + 4x_{2,1} + 6x_{2,2} + 3x_{2,3} + 1x_{2,4}$

Subject to Constraints:

Binary Variables Constraint

$x_{s,t}, y_s \in [0, 1]$

Target/Site Equality Constraint

`Each Target must be drilled from only one Site.`

$x_{1,1} + x_{2,1} = 1$ (Target 1)

$x_{1,2} + x_{2,2} = 1$ (Target 2)

$x_{1,3} + x_{2,3} = 1$ (Target 3)

$x_{1,4} + x_{2,4} = 1$ (Target 4)

Site Preparation for Target Assignment Inequality Constraint

`Can only assign Target to a Site that is prepared.`

$x_{1,t} \leq y_1$ for $ t [1, 2, 3, 4]$

$x_{2,t} \leq y_2$ for $ t [1, 2, 3, 4]$

[5]:
```
""" Problem 4 Setup """
# objective func, format as 1D array
c = [
    2, 1, 8, 5,   # drilling s=1 to t= 1 thru 4
    4, 6, 3, 1,   # drilling s=2 to t= 1 thru 4
    5, 6          # prep costs site 1, site 2
]

# Tar/site Equ Constraint
A_eq = [
    [1,0,0,0,1,0,0,0,0,0],  # t=1
    [0,1,0,0,0,1,0,0,0,0],  # t=2
    [0,0,1,0,0,0,1,0,0,0],  # t=3
    [0,0,0,1,0,0,0,1,0,0],  # t=4
]
b_eq = 4*[1] # RHS equality

# Site prep for Target inequality constraint
A_ub = [
    # site 1
    [1,0,0,0,0,0,0,0, -1,0],  # x_1,1 <= y_1
    [0,1,0,0,0,0,0,0, -1,0],  # x_1,2 <= y_1
    [0,0,1,0,0,0,0,0, -1,0],  # x_1,3 <= y_1
    [0,0,0,1,0,0,0,0, -1,0],  # x_1,4 <= y_1
    # site 1
    [0,0,0,0,1,0,0,0,0, -1],  # x_2,1 <= y_2
    [0,0,0,0,0,1,0,0,0, -1],  # x_2,2 <= y_2
    [0,0,0,0,0,0,1,0,0, -1],  # x_2,3 <= y_2
    [0,0,0,0,0,0,0,1,0, -1],  # x_2,4 <= y_2
]
b_ub = 8*[0] # RHS inequaliy

# Binary vars constraint
bounds = 10*[(0, 1)]
```

5 (10 points) Each day at the Graphic Arts Co. the press operator is given a list of jobs to be done during the day. He must determine the order in which he does the jobs based on the amount of time it takes to change from one job setup to the next. He will arrange the jobs in an order which minimizes the total setup time. Assume that each day he starts the press from a rest state and returns it to that state at the end of the day. Suppose on a particular day he must do six jobs for which he estimates the changeover times given in Table 5. What schedule of jobs should the operator use? (Feel free to use cij to represent the coefficients in the table if necessary.)

Table 5

	To job j (min)						
From job i	j=1	j=2	j=3	j=4	j=5	j=6	rest
i=1	0	10	5	15	10	20	5

Answer:

Binary Decision Variables:

Let $x_{i,j} \in 0, 1$, where...

$x_{i,j} == 1$: if press operator goes from job i to j.

$x_{i,j} == 0$: if press operator does not go from job i to j.

for $i \neq j$, where $i, j \in \text{rest}, 1, 2, 3, 4, 5, 6$.

Elimination Variables For Subtours:

For each job $i \in 2, 3, 4, 5, 6$, let $u_i \in \mathbb{Z},$ such that $1 \leq u_i \leq n$ for $i \in 2, 3, 4, 5, 6$ (where $n = 6$) be an auxiliary variable used to eliminate subtours.

Objective Function:

Let $x_{i,j} \in 0, 1 = 1$ when press operator goes from job i to j, and let $c_{i,j} =$ the time cost for operator to change from job i to j.

Job 1: $> 10x_{1,2} + 5x_{1,3} + 15x_{1,4} + 10x_{1,5} + 20x_{1,6} + 5x_{1,rest}$

Job 2: $> 10x_{2,1} + 12x_{2,3} + 8x_{2,4} + 15x_{2,5} + 6x_{2,6} + 7x_{2,rest}$

Job 3: $> 5x_{3,1} + 12x_{3,2} + 18x_{3,4} + 9x_{3,5} + 14x_{3,6} + 6x_{3,rest}$

Job 4: $> 15x_{4,1} + 8x_{4,2} + 18x_{4,3} + 11x_{4,5} + 13x_{4,6} + 4x_{4,rest}$

Job 5: $> 10x_{5,1} + 15x_{5,2} + 9x_{5,3} + 11x_{5,4} + 17x_{5,6} + 8x_{5,rest}$

Job 6: $> 20x_{6,1} + 6x_{6,2} + 14x_{6,3} + 13x_{6,4} + 17x_{6,5} + 9x_{6,rest}$

Rest: $> 5x_{rest,1} + 7x_{rest,2} + 6x_{rest,3} + 4x_{rest,4} + 8x_{rest,5} + 9x_{rest,6}$

Providing us with a full **objective function** of:

Minimize Time (total) $= 10x_{1,2} + 5x_{1,3} + 15x_{1,4} + 10x_{1,5} + 20x_{1,6} + 5x_{1,rest} + 10x_{2,1} + 12x_{2,3} + 8x_{2,4} + 15x_{2,5} + 6x_{2,6} + 7x_{2,rest} + 5x_{3,1} + 12x_{3,2} + 18x_{3,4} + 9x_{3,5} + 14x_{3,6} + 6x_{3,rest} + 15x_{4,1} + 8x_{4,2} + 18x_{4,3} + 11x_{4,5} + 13x_{4,6} + 4x_{4,rest} + 10x_{5,1} + 15x_{5,2} + 9x_{5,3} + 11x_{5,4} + 17x_{5,6} + 8x_{5,rest} + 20x_{6,1} + 6x_{6,2} + 14x_{6,3} + 13x_{6,4} + 17x_{6,5} + 9x_{6,rest} + 5x_{rest,1} + 7x_{rest,2} + 6x_{rest,3} + 4x_{rest,4} + 8x_{rest,5} + 9x_{rest,6}$

Which can be written in simpler form as:

Minimize Time $\Sigma_{i \in [rest,1,2,3,4,5,6]} \Sigma_{j \in [rest,1,2,3,4,5,6], j \neq i} c_{i,j} x_{i,j}$

Subject to Constraints:

Job Start Constraint

`Each job has to be entered one time exactly.`

$\Sigma_{i \neq j} x_{i,j} = 1$ for all $j \in [1, 2, 3, 4, 5, 6]$

Job Departure Constraint

`Each Job has to be left exactly one time.`

$\Sigma_{j \neq i} x_{i,j} = 1$ for all $i \in [1, 2, 3, 4, 5, 6]$

Resting Start Constraint

`Operator must start in rest state.`

$\Sigma_{j=1}^{6} x_{rest,j} = 1$

Resting End Constraint

`Operator must end in rest state.`

$\Sigma_{i=1}^{6} x_{i,rest} = 1$

Elemination Variable Constraint

`Each Job is entered and exited exactly one time. Set of Jobs must be visited exactly once witho`

$u_i - u_j + M x_{i,j} \leq M - 1$ for all $i \neq j$, where M is a large constant (i.e. $M == n$, the total number of jobs).

[6]:
```
""" Problem 5 Setup """

# obj func
c = [
    5, 7, 6, 4, 8, 9, # rest to 1-6
    10, 5, 15, 10, 20, 5, # job 1 to 2,3,4,5,6,0
    10, 12, 8, 15, 6, 7, # job 2to 1,3,4,5,6,0
    5, 12, 18, 9, 14, 6, # job 3 to 1,2,4,5,6,0
    15, 8, 18, 11, 13, 4, # job 4 to 1,2,3,5,6,0
    10, 15, 9, 11, 17, 8, # Job 5 to 1,2,3,4,6,0
    20, 6, 14, 13, 17, 9 # job6 to 1,2,3,4,5,0
```

```
]

def get_var_index(i,j):
    """ Helper function to get variable index. """
    if i == j:
        raise ValueError("No vars i == j")
    else:
        ind = 0
        for a in range(7):
            for b in range(7):
                if a != b:
                    if a == i and b == j:
                        return ind
                    else:
                        ind += 1
        raise ValueError("Bad index i=", i," j=", j)

# equalitys
A_eq = []
jobStartEnd = 2*6*[1] # 6 jobs, entering and exit(*2)
restStartEnd = [1, 1] # rest at start &end
b_eq = jobStartEnd + restStartEnd

# populating A_eq, b_eq
# Job Start
for j in range(1, 7):
    row = len(c)*[0]
    for i in range(7):
        if i != j:
            ind = get_var_index(i, j)
            row[ind] = 1
    A_eq.append(row)

# job end
for i in range(1, 7):
    row = len(c)*[0]
    for j in range(7):
        if j != i:
            ind = get_var_index(i, j)
            row[ind] = 1
    A_eq.append(row)

# resting Start
row = len(c)*[0]
for j in range(1, 7):
    ind = get_var_index(0, j)
    row[ind] = 1
```

```
A_eq.append(row)

# rest end
row = len(c)*[0]
for i in range(1, 7):
    idx = get_var_index(i, 0)
    row[idx] = 1
A_eq.append(row)

# binary const
bounds =  len(c)*[(0, 1)]
```